FileDrop

v1.0

Large File Transfer System

LAN · Internet · Up to 100 GB · Zero Dependencies

Submitted by – Chandrashekhar Tyagi
chandershekhar8433@gmail.com

Python 3.6+ | Browser UI | No pip install | LAN + Internet

Technical Presentation · April 2026

OVERVIEW

What We'll Cover

01 **The Problem**
Why existing tools fail for large files

02 **What is FileDrop?**
Architecture, tech stack, and key features

03 **How It Works**
3-phase protocol, chunking, and data flow

04 **System Flowchart**
Complete end-to-end flow visualised

05 **File Structure**
14-file codebase breakdown

06 **Performance**
Speed benchmarks and memory usage

07 **Networking**
LAN vs Internet — how both modes work

08 **Security**
Current posture and planned hardening

09 **Roadmap**
10 planned enhancements with priorities

10 **Quick Start**
Running FileDrop in under 60 seconds

PROBLEM

The Large File Transfer Problem

Every common method breaks down at large file sizes

Tool	Size Limit	Speed	Account?
Email (Gmail)	25 MB	N/A	Yes
WeTransfer Free	2 GB	5–15 MB/s	No
Google Drive	15 GB free	10–30 MB/s	Yes
USB Drive	Drive size	100 MB/s	No
FTP Server	Unlimited	100 MB/s	Setup needed
FileDrop ✓	100 GB+	100 MB/s LAN	None

SOLUTION

What is FileDrop?

FileDrop is a zero-dependency Python HTTP server that lets any two computers transfer files up to 100 GB directly — no cloud, no accounts, no size limits. The receiver runs one Python command. The sender only needs a browser.

Zero Dependencies

Pure Python stdlib.
No pip install required.

Chunked Transfer

4 MB pieces ensure
100 GB never fails.

LAN + Internet

Local network or
global via ngrok.

Clean Architecture

14 files, each with
one clear purpose.

STATS

FileDrop by the Numbers

100 GB
Max file size per transfer

4 MB
Chunk size (streaming pieces)

0
External dependencies (pip)

14
Source files in total

~900
Lines of total code

100 MB/s
Peak speed on Gigabit LAN

2 s
Receiver refresh interval

3
Transfer phases (init/chunk/complete)

PROTOCOL

The 3-Phase Upload Protocol

Every file transfer follows a strict, predictable sequence

Phase 01

Initialisation

- Generate unique file_id
- POST /api/upload/init
- Server creates temp dir
- TransferStore entry added

```
POST /api/upload/init
{ file_id, filename, size,
  total_chunks }
```

Phase 02

Chunk Upload Loop

- file.slice(i × 4MB)
- POST /api/upload/chunk
- Server writes chunk_N
- record_chunk() updates state

```
FOR i = 0 to N-1:
  slice → multipart POST
  server writes chunk_i
  progress updated
```

Phase 03

Completion

- POST /api/upload/complete
- assemble_chunks() runs
- Chunks concatenated → file
- active → completed store

```
POST /api/upload/complete
assemble chunk_000000..N
delete temp chunks
✓ file ready
```

Why Chunking Solves Everything

Without Chunking

- Entire 100 GB loaded into browser memory
- One TCP error = restart from zero
- Server holds full file in RAM
- Browser freezes on large files
- Progress % is meaningless until done

100 GB — ONE REQUEST

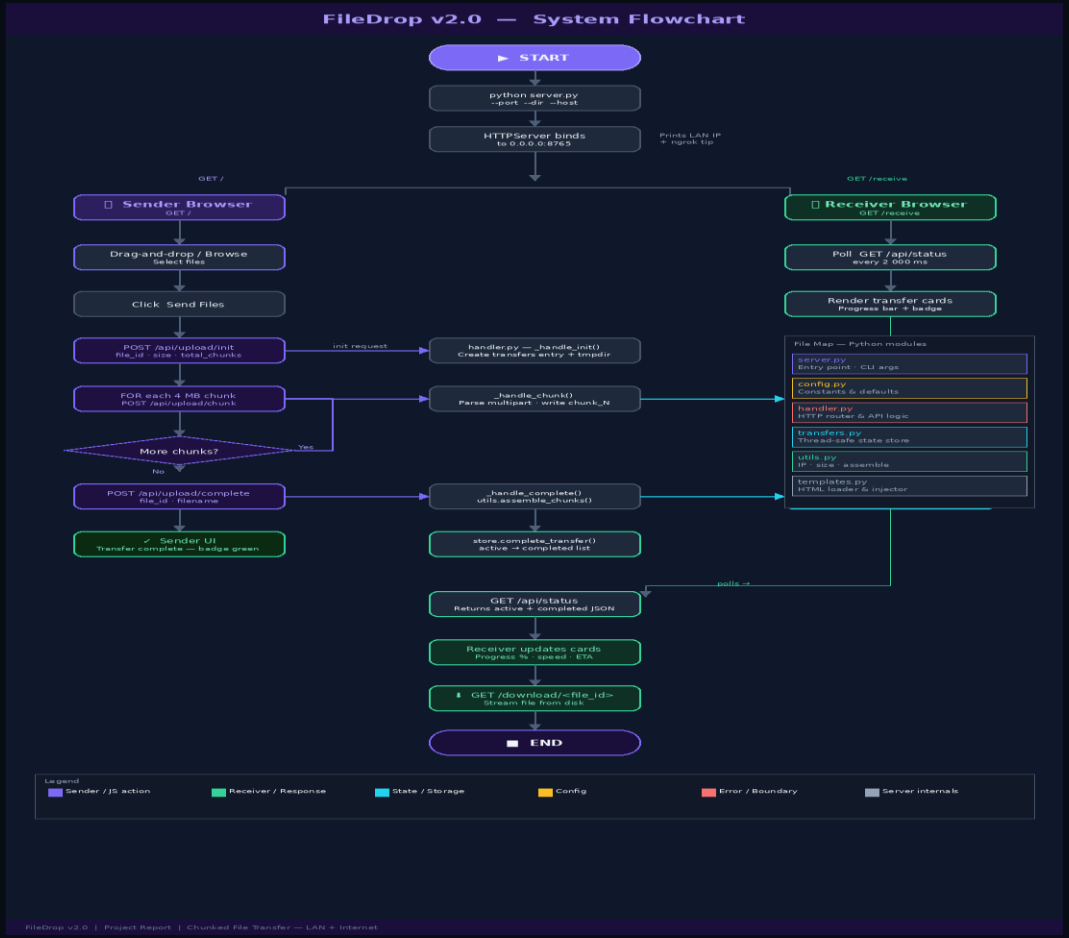
With 4 MB Chunks

- Only 4 MB in memory at any moment
- Network error? Retry just that chunk
- Server writes each chunk to disk immediately
- Browser stays responsive throughout
- Live speed, progress & ETA always accurate

Each 4 MB

FLOWCHART

System Flowchart



1	**Server starts** Python binds to port 8765, detects LAN IP
2	**Pages served** Sender opens /, Receiver opens /receive
3	**Init + chunks** Sender sends metadata then 4 MB slices
4	**State tracked** TransferStore records every chunk received
5	**Assembly** All chunks stitched into final file on disk
6	**Download ready** Receiver sees complete badge + Download button

STRUCTURE

14-File Codebase — Clean Separation

```
filedrop/
├── server.py
├── config.py
├── handler.py
├── transfers.py
├── utils.py
├── templates.py
├── templates/
│   ├── sender.html
│   └── receiver.html
└── static/
    ├── css/common.css
    ├── css/sender.css
    ├── css/receiver.css
    ├── js/common.js
    ├── js/sender.js
    └── js/receiver.js
```

Python — Entry & Config
`server.py, config.py`

Python — Routing & State
`handler.py, transfers.py`

Python — Helpers & HTML
`utils.py, templates.py`

HTML — Page Structure
`sender.html, receiver.html`

CSS — Styling & Themes
`common, sender, receiver .css`

JS — Logic & UI Updates
`common, sender, receiver .js`

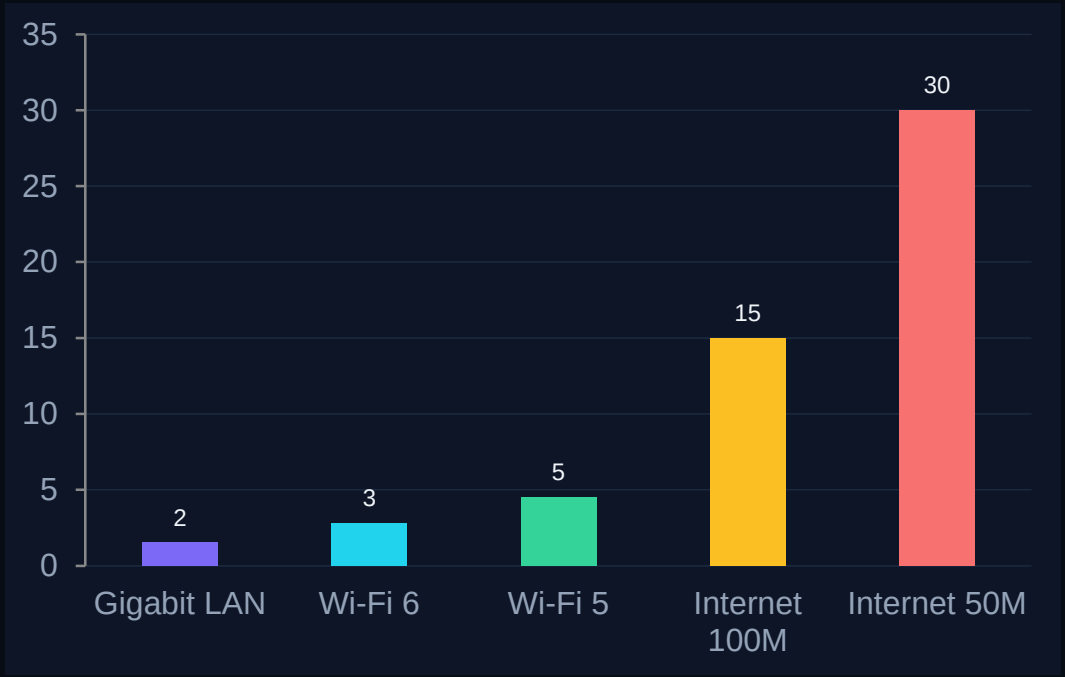
API

API Endpoint Reference

GET	/	Returns sender.html — the upload interface
GET	/receive	Returns receiver.html — the live progress UI
GET	/static/<path>	Serves CSS and JS files with 1hr cache
GET	/api/status	JSON: active + completed transfers (polled every 2s)
GET	/download/<file_id>	Streams assembled file to browser as attachment
POST	/api/upload/init	Registers transfer: file_id, filename, size, total_chunks
POST	/api/upload/chunk	Receives 4 MB piece as multipart/form-data
POST	/api/upload/complete	Triggers assembly of all chunks into final file

PERFORMANCE

Transfer Speed Benchmarks



Minutes to transfer 10 GB

Memory Usage

File Size	Naive	FileDrop
1 GB file	1 024 MB	**4 MB**
10 GB file	10 240 MB	**4 MB**
100 GB file	CRASH ✸	**4 MB**

FileDrop always uses exactly 4 MB regardless of file size

NETWORKING

Two Network Modes

LAN Mode

Same Wi-Fi or Ethernet network

- No internet required
- Speeds up to 100 MB/s
- Works during internet outages
- Use 192.168.x.x:8765 directly
- Wired = 3× faster than Wi-Fi

Internet Mode

Different networks worldwide

- Run ngrok http 8765 in terminal
- Get public https://xxxx.ngrok.io URL
- Share URL — works anywhere globally
- ngrok provides HTTPS encryption
- Free tier is sufficient for transfers

FAQ

3 Key Questions Answered

Q1: Can we do this WITHOUT a server?

FileDrop's 'server' is just Python running on YOUR machine — not a cloud. Files never leave your computer. For truly serverless: use WebRTC (wormhole.app), Shared Folder, or AirDrop on LAN.

Q2: Can we do this WITHOUT the internet?

YES — this is the primary use case! Both computers on same Wi-Fi/Ethernet → open http://192.168.x.x:8765 → done. Zero internet required. Even works with two laptops connected by a single cable.

Q3: Can we run the server permanently and use it whenever?

YES — host on a VPS (DigitalOcean ~$5/mo), a Raspberry Pi at home, or use Cloudflare Tunnel for a permanent free URL. Add systemd for auto-start on reboot. Needs password protection first.

HOSTING

Running FileDrop Permanently

FREE

Option A — Home Machine (Free)

- Old laptop or Raspberry Pi
- Run server.py with systemd service
- Use DuckDNS for free dynamic domain
- Pair with Cloudflare Tunnel for HTTPS

$5/mo

Option B — Cloud VPS ($4–6/mo)

- DigitalOcean / Hetzner / Linode
- Fixed public IP — URL never changes
- systemd for auto-start on reboot
- Needs password protection added first

FREE

Option C — Cloudflare Tunnel (Free)

- cloudflared tunnel --url http://localhost:8765
- Permanent named subdomain URL
- Auto-reconnects if machine reboots
- Free tier, no port forwarding needed

```
sudo systemctl enable filedrop && sudo systemctl start filedrop    # Auto-start on every reboot
```

SECURITY

Security Posture

No Authentication (Current)

⚠ Planned

Anyone with the URL can upload/download. Fine for LAN; add password before internet hosting.

No Encryption on LAN (Current)

⚠ LAN Only

HTTP only on local network. ngrok provides HTTPS automatically for internet transfers.

File Name Sanitisation

⚠ Planned

Files are sandboxed under UPLOAD_DIR/<file_id>/ which prevents path traversal. Explicit sanitisation planned.

Memory Safety

✓ Done

4 MB chunk limit means no memory overflow. Files streamed to disk immediately.

Thread Safety

✓ Done

All state mutations protected by threading.Lock. No race conditions on concurrent uploads.

Disk Isolation

✓ Done

Each transfer isolated in its own UUID subdirectory. No inter-transfer contamination.

ROADMAP

Roadmap — 10 Planned Enhancements

Password Protection
High

HTTP Basic Auth or URL token

SHA-256 Checksum
High

Web Crypto API verification

Resume Interrupted Transfers
High

Skip already-received chunks

Parallel Chunk Uploads
Medium

Promise.all() for 3–5× speed

QR Code on Receiver Page
Medium

One scan to open sender URL

Transfer History (JSON)
Medium

Persist metadata across restarts

HTTPS / TLS Support
Medium

ssl.wrap_socket() in server.py

Docker Container
Medium

One-command cloud deployment

Multi-Receiver Broadcast
Low

1 sender → N receivers

Speed Throttle Option
Low

--max-speed flag in config

QUICK START

Up and Running in 60 Seconds

LAN Transfer

1. On RECEIVER:
   `python server.py`
2. Terminal prints:
   `http://192.168.1.x:8765/`
3. RECEIVER opens:
   `/receive in browser`
4. SENDER opens:
   `the printed URL`
5. Sender drags files &
   `clicks Send Files`
6. Receiver sees progress,
   `clicks ⬇ Download`

Internet Transfer

1. On RECEIVER — Terminal 1:
   `python server.py`
2. On RECEIVER — Terminal 2:
   `ngrok http 8765`
3. ngrok prints:
   `https://xxxx.ngrok.io`
4. Share with SENDER:
   `https://xxxx.ngrok.io/`
5. RECEIVER opens:
   `https://xxxx.ngrok.io/receive`
6. Transfer proceeds exactly
   `the same as LAN`

TECH STACK

Technology Stack

Backend

Python 3.6+

http.server, socket, threading, pathlib, json, mimetypes — all stdlib

HTTPServer

BaseHTTPRequestHandler — single file, no frameworks

threading.Lock

Thread-safe state management for concurrent transfers

Frontend

HTML5

File API, FormData, Fetch API, drag-and-drop events

CSS3

Custom properties, keyframe animations, CSS Grid, flexbox

Vanilla JS

No frameworks — pure ES6+. file.slice(), FormData, fetch()

SUMMARY

Why Choose FileDrop?

Zero Setup

Receiver runs 1 command.
Sender needs only a browser.
Nothing to install.

Constant Memory

4 MB RAM regardless of
file size. No crashes.
No OOM errors.

Clean Codebase

14 files, one purpose each.
Easy to read, modify,
and extend.

Truly Universal

LAN or internet.
Any OS, any browser.
Zero dependencies.

Direct Transfer

No cloud intermediary.
File goes machine→machine.
You control your data.

Extensible

Add auth, checksums, QR,
or Docker — architecture
supports all of it.

Thank You

FileDrop v2.0 — Large File Transfer System

python server.py | 100 GB files | Zero dependencies | LAN + Internet

Questions? Open the project and explore:

```
python server.py  →  http://localhost:8765/
```

Built with Python · Zero Dependencies · Open Architecture · April 2026